北京郵電大學
Beijing University of Posts and Telecommunications

2023110949

硕士研究生学位论文中期报告

学　　号: 2023110949

姓　　名: 段欣悦

学 生 类 别: 学硕(全日)

学　　院: 计算机学院（国家示范性软件学院）

学科/专业(代码): 计算机科学与技术(081200)

研究方向: 网络服务、管理与安全

导师姓名: 王智立

攻 读 学 位: 工学硕士

北京邮电大学

2025 年 10 月 16 日

论文题目	面向满意度和可靠性的云服务分级工作流在线调度方法		
论文类型	应用研究	选题来源	学校自选项目
开题日期	2024-11-28	是否开题题目	是
论文开始日期	2024-11-28	报告日期	2025-10-29
报告地点	科研楼 508	报告时间	上午 13:00
已提交月报次数	9	已提交年报次数	0

一、研究内容简介

1.1. 选题背景

随着信息技术的飞速发展，云计算凭借其按需供给、弹性伸缩的资源模式，已成为支撑现代企业运营和各类互联网应用的核心基础设施。在此背景下，云工作流——即由一系列具有数据依赖关

系的任务构成的复杂应用——被广泛部署于科学计算、大数据分析与业务流程处理等场景。这些工作流任务通常动态到达云平台，且对完成时间、资源消耗及服务可靠性存在差异化需求，这使得云环境下的动态工作流在线调度成为一项关键且极具挑战性的研究课题。

高效的调度策略是以提升云平台资源利用率、降低运营成本与保障用户服务质量为核心。然而，实际云环境具有高度动态与不确定特性，具体表现为任务执行时间波动、网络传输延迟变化及资源节点性能起伏。此外，企业工作流天然具备服务等级差异：例如，实时交易处理、紧急数据分析等高优先级工作流对时效性与可靠性有严格要求；而批量报表生成、历史数据备份等低优先级工作流则更注重执行的经济性，对服务中断具有相对较高的容忍度。

在成本优化方面，现有研究多聚焦于资源分配与能效提升。Jamil,B.等[1]提出一个基于深度强化学习的在线资源分配和任务调度算法（IRATS），它在车辆雾计算网络中优化任务的执行，减少延迟和提高任务完成率，从而有效降低了成本和能耗。M. Fan 等[2]提出了一个名为 PACP-HEFT 的算法，通过优先级调整和关键任务优化来解决云计算中预算受限的工作流调度问题，以最小化总执行时间，同时确保成本控制在预设的预算范围内。G. Fa 等[3]提出了一种混合工作流调度算法（OHDS），通过最小化任务能耗和最大化服务资源利用率，同时满足工作流任务的子截止日期和数据依赖性约束，有效解决了云计算环境中成本优化问题。Mohammed Maray 等[4]提出一种基于优先级感知调度的依赖任务卸载策略，在移动边缘网络中优化了任务的执行时间，从而降低了成本，提高了任务满足率和系统吞吐量。L. Ye 等[5]提出了一种名为 SMWDSA 的动态调度算法，用于在云环境中调度具有截止日期约束的随机多工作流，以优化工作流调度成本。J. Jin 等[6]提出了一种基于动态规划和强化学习的联合调度算法，用于优化具有随机供应和处理速率限制的可推迟需求和存储系

统的调度问题，以最小化预期总成本。Sharif 等[7]提出了一种基于优先级的边缘计算任务调度和资源分配（PTS-RA）机制，用于健康监测系统，能够根据不同任务的紧急程度分配不同优先级，并优化决策以减少处理时间和带宽消耗。J. X. Liao 等[8]提出了一种基于服务紧急优先级的资源分配和任务调度优化方案，并设计了动态优先级任务调度算法（DPTSA），以最小化系统总延迟并确保高优先级服务的最小延迟。Bebortta 等人[19]提出了一种面向云雾混合网络的计算卸载机制，通过构建基于 M/M/1 排队模型的动态资源选择策略，在考虑任务队列等待时间的基础上，有效权衡了实时负载限制与云服务使用成本，显著降低了处理开销；Reddy 等人[20]设计了资源感知调度算法 RAS，通过分析资源日志、工作负载特征与价格等因素，动态映射用户请求与可用资源，在提升资源分配效率的同时实现了成本优化；Lei 等人[21]则融合 FFD 与贪心算法，提出一种启发式云资源部署与调度策略，通过合理规划虚拟机的服务器类型选择与迁移机制，显著降低了企业运营成本；Hlaing 等人[22]针对虚拟化环境提出一种静态独立任务调度方法，通过综合考虑处理能力、成本及处理单元数量等资源属性，将任务按其指令长度分组调度，在保证执行时间的基础上有效降低了总体执行成本。这些研究从不同维度推进了云环境中成本优化的方法体系，为本文在分级工作流调度的研究提供了重要参考。

当前云服务环境中的工作流调度策略普遍优先保障高优先级工作流的快速执行，以满足关键任务的时效性需求。然而，此类策略往往忽视低优先级工作流的资源获取机会，导致其长期处于等待状态，甚至引发“任务饿死”问题。为系统缓解该矛盾，本文提出一种面向服务满意度的云服务分级工作流在线调度方法。该方法以服务满意度为核心优化目标，旨在综合权衡不同优先级工作流在时间成本与资源成本方面的差异化需求。通过综合考虑工作流优先级、时间窗口约束与任务间隔等关键属性，构建

更为合理的任务排序机制，在保障高优先级任务获得及时响应的同时，兼顾低优先级任务的执行推进，从而有效缓解资源分配不均问题。

在任务分配层面，本文设计了差异化的资源调度机制，依据不同优先级任务对时间与资源成本的敏感程度，动态调整资源分配策略。具体而言，对高优先级任务优先配置高性能资源以缩短其等待与执行时间；对低优先级任务则采用成本节约型资源分配方案，以控制整体执行开销。为进一步提升用户满意度，引入基于任务完成状态的激励与约束机制：对按时完成的任务给予正向奖励，激励云服务提供方持续优化调度；对超时任务实施相应惩罚，推动系统服务质量的持续改进。该机制不仅有助于提升任务执行效率，也有助于增强用户对云服务系统的整体信任与满意度。

基于上述文献分析，本课题第一个理论研究部分面临的挑战如下：

（1）缺乏对分级成本需求的精细考量：现有研究大多未明确区分不同优先级任务流对时间成本和资源成本的差异化敏感度，难以在保障高优先级任务时效性的同时，为低优先级任务实现最佳成本效益。

（2）动态环境下的平衡难题：在动态负载环境下，如何构建有效的在线调度机制，既确保关键任务优先执行，又避免低优先级任务长期资源匮乏

（3）满意度量化与激励缺失：缺乏一个融合时间成本与资源成本的多维度满意度评估体系，以及与之配套的动态激励与惩罚机制，难以系统引导调度决策向提升整体满意度方向演进。

为应对上述挑战，本课题提出一种面向服务满意度的云服务分级工作流在线调度方法。其核心内容包括：

（1）设计一种任务紧急性评估指标，综合考量任务优先级、时间窗口与间隔约束等属性，作为

在线调度中任务排序的依据。

（2）构建差异化任务分配机制，为不同优先级任务定制资源分配策略，实现时间与资源成本的有效平衡。

（3）引入综合服务满意度指标，该指标依据任务流完成情况动态计算奖励与惩罚值，从而将时间效率与资源节约统一为一个可优化的目标函数，激励调度系统持续提升整体用户满意度。

在可靠性研究方面，学者们重点关注服务稳定性与任务完成保障。J. Liu 等[9]提出了一种可靠性增强的任务卸载算法 RETO 和基于差分进化的卸载算法 DETO，用于在移动边缘计算环境中最小化物联网应用的带宽消耗，同时最大化这些应用的可靠性水平。M. Zhao 等[10]提出了一种基于深度 Q 学习的无人机群中深度神经网络分割策略，旨在在满足延迟和可靠性要求的同时最小化多无人 机协同推理任务的能耗。Wang, Z.等[11]提出了一种基于深度学习网络的智能高光谱目标检测方法，该方法基于残差网络结构并加入了注意力机制，以提高物联网车辆应用中的检测准确性和实时性。L.Chen 等[12]提出了一种名为 REMR 的可靠性评估方法，用于在时间约束下的动态边缘计算网络中评估分布式和协作服务部署策略的可靠性。K. Peng 等[13]提出了一种基于 Stackelberg 博弈的可靠性感知的主动卸载算法，用于移动边缘计算环境中优化任务卸载决策，并采用交替方向乘子法算法来推导边缘服务器提供商和移动用户的最优策略。L.Dong 等[14]提出了一种针对多级边缘计算系统中的可靠性感知计算卸载与分配算法，通过将问题转化为 n 重整数规划问题并设计了一种基于阴影组件的可靠性方案，以优化多用户多服务器环境下的任务分配策略，降低系统延迟和能耗。B.Ju 等[15]提出了一种基于任务导向的三阶段全周期监控机制的高可靠边缘计算架构，用于动态共享移动终端的计算资源，并确保系统的可靠性和任务的高效执行。R.Huang 等[16]提出了一种基于 Lyapunov

优化框架的联合功率分配和任务复制算法 PATRLF，用于在 NOMA 赋能的车辆边缘计算环境中优化可靠性敏感服务。Meng 等人[23]从服务视角提出了一种云计算系统可靠性建模方法，将云服务划分为请求处理与执行两个阶段，并结合自主调度、异常检测与学习优化机制，有效提升了公有云环境下的服务可靠性；Tang 等人[24]则关注成本与可靠性的权衡，提出一种基于主备容错机制的启发式作业调度算法，通过结合响应时间松弛因子与服务实例故障预测，在控制服务成本的同时实现了较高的系统可靠性；Gill 与 Buyya[25]系统性地构建了云失败管理的分类学与概念模型，为可靠性研究提供了理论框架与分析基础；Alsenani 等人[26]针对志愿者云中边缘资源的高度动态特性，提出了一种基于随机模型的可靠性评估系统 SaRa，通过细粒度追踪任务级行为与特征，提升了不可信环境下资源可靠性的评估精度。这些工作为可靠性优化奠定了重要基础，但在支持分级工作流的差异化可靠性需求方面仍存在明显局限，例如多数方法未考虑不同优先级任务在容错策略、备份资源分配与可靠性目标上的区别化处理，也缺乏对工作流内部依赖结构对可靠性影响的动态建模。

尽管已有研究取得显著进展，但在支持动态任务流中不同优先级任务的差异化需求方面仍存在不足。特别是在系统过载情况下，如何有效平衡高低优先级任务的执行，并满足其特定的成本与可靠性需求，仍是当前研究面临的重要挑战。现有方法往往未能充分考虑任务流的动态特性与优先级差异，导致在实际云环境中的调度效果受到限制。

基于对现有研究的深入分析，本课题识别出第二个研究点的以下关键理论挑战：

（1） 可靠性需求与任务优先级的解耦问题：现有调度策略普遍缺乏任务关键性与服务实例可靠性之间的智能匹配机制，导致高优先级任务可能被分配到低可靠实例，而低优先级任务却占据优质可靠性资源。这种资源错配不仅降低了系统整体效率，更难以满足关键任

务的可靠性要求。

（2） 静态策略与动态环境的适配性不足：大多数现有方法依赖于固定的可靠性阈值或预设的冗余策略，缺乏对云环境动态特性的有效响应。面对服务实例稳定性波动、任务负载变化等实时因素，传统静态策略难以实现可靠性资源的自适应调整与优化分配。

（3） 故障恢复与前瞻调度的协同机制缺失：现有研究往往将故障处理视为独立于调度的应急机制，未能将重配置策略与前期调度决策进行深度整合。特别是在保障高优先级工作流快速恢复方面，缺乏与任务分级机制紧密耦合的弹性容错方案。

针对上述挑战，本课题提出一种基于强化学习的云服务分级工作流可靠性调度框架。该方法的核心在于构建一个能够感知系统状态、学习调度经验并动态优化决策的智能调度系统，具体通过以下四个层面实现：

（1）建立智能化的优先级驱动的可靠性资源分配机制：设计基于强化学习的动态可靠因子分界值计算模型，通过实时分析任务特征与服务实例状态，建立任务优先级与实例可靠性之间的最优映射关系，确保高关键性任务被精准导向高稳定性实例，实现可靠性资源的按需分配。

（2）构建多目标协同的自适应评估体系：提出融合任务执行效率、经济成本与实例稳定性的多维适配度指标，利用强化学习算法在线调整各目标权重，在保障核心可靠性需求的同时，实现调度效率与资源利用的综合优化。

（3）开发故障感知的弹性重配置策略：构建故障快速响应机制，在检测到服务实例失效时，能够结合任务关键性等级与系统实时状态，智能生成最优恢复方案，最大限度降低故障对高优先级工作流的影响。

（4）实现持续进化的调度决策能力：通过强化学习的在线学习机制，使调度系统能够从历史调度经验中不断提炼优化策略，自适应环境变化，逐步提升在复杂云环境下的长期调度性能与鲁棒性。

1.2. 研究内容

在云服务环境中，设计一种能够协同优化用户服务满意度与系统可靠性，并能够有效区分与处理不同优先级工作流的智能在线调度方法，已成为一个亟待解决的关键科学问题。

本课题旨在研究面向满意度和可靠性的云服务分级工作流在线调度方法，具体包括以下两个研究方向：

1.2.1 提出面向满意度的云服务分级工作流在线调度方法

通过定义“任务紧急性”与“服务选择收益”等量化指标，实现对不同优先级任务的差异化调度：为高优先级任务优先分配资源以保证时效性，同时为低优先级任务筛选资源成本更优的执行路径。在此基础上，引入基于任务完成状态的动态奖惩机制，综合评估整体服务满意度，从而在保障关键任务快速响应的同时，兼顾普通任务的执行效率与经济性，实现服务满意度指标的系统性优化。

任务紧急性指标： 该指标用于对就绪任务集中的任务进行排序，并依序调度至相应服务实例。其计算综合考虑任务优先级、时间余量及预计最早开始时间等因素。在时间余量计算中，本研究提出一种“向上排序秩”方法，以获取聚簇最晚完成时间。聚簇结构能够识别子任务在工作流拓扑中的层级，使同一簇内任务在调度时具有更高灵活性，同时确保不延误后续任务执行。

服务选择收益指标： 该指标用于在任务调度过程中选择最适合的服务实例。其设计兼顾高优先级任务对时间成本的敏感性，以及低优先级任务对资源成本的经济性需求。系统根据任务优先级动态调整评估权重，以实现服务实例的精准匹配。

本节将问题建模为约束优化问题。设 W 为动态到达的分级工作流集合。本研究的目标是找到一个可行的调度策略，在满足不同优先级工作流成本约束的同时，最大化整体服务满意度，记为 Satisfaction(W)。服务满意度指标对提前完成的工作流给予奖励，奖励值由提前完成时间与资源成本节约比例共同决定；对未按时完成的工作流实施惩罚，惩罚基于已消耗资源成本与平均成本的比值计算。

对于已完成的工作流 (f_i = 1)，其奖励 R_i 综合考量了时间提前量与成本节约率：

$$R_i = \delta_{priority} * \left[\left(1 - \gamma_{priority}\right) * \frac{ddl_i - T_{end_i}}{ddl_i - T_{start_i}} + \gamma_{priority} * \left(1 - \frac{\sum_{j=1}^{|S|_i} P(\mu)_{i,j} * ret_{i,j}}{\sum_{j=1}^{|S|_i} Cost_{max_{i,j}}} \right) \right] \quad (1)$$

对于未完成的工作流 (f_i = 0)，其惩罚 P_i 基于其已执行任务消耗的资源成本进行计算：

$$Cost_{avg_{i,j}} = \frac{\sum_{k=1}^{|\mu|} P(\mu)^k * pet_{i,j}^k}{|\mu|} \quad (2)$$

$$P_i = \delta_{priority} * \frac{\sum_{j=1}^{|S|_i} P(\mu)_{i,j} * ret_{i,j}}{\sum_{j=1}^{|S|_i} Cost_{max_{i,j}}} \quad (3)$$

最终，系统总体服务满意度定义为：

$$Reward = \sum_{n=1}^{n=|i|} f_i * R_i \quad (4)$$

$$Punish = \sum_{n=1}^{n=|i|} （ f_i - 1） * P_i \tag{5}$$

$$Satisfication = Reward + Punish \tag{6}$$

该模型通过参数$\delta_{priority}$与$\gamma_{priority}$实现了对不同优先级工作流的差异化处理，并有效平衡了时间与成本两个关键目标。

1.2.2 提出面向可靠性的云服务分级工作流在线调度方法

引入动态可靠性因子分界值机制，依据系统实时负载与任务关键性自适应调整资源分配策略，优先将高可靠服务实例分配给高优先级任务。调度过程中，系统根据就绪任务池的规模动态选择强化学习调度器或强化学习与模拟退火混合优化调度器。算法实时计算可靠性感知的动态分界值，并据此进行优先级自适应分配，确保高优先级任务匹配高稳定性实例，低优先级任务匹配成本优化实例。

同时，设计多目标奖励函数，同步优化时间、成本与可靠性指标，以在复杂云环境中实现分级工作流的高效在线调度，显著提升系统容错经济效能。系统还实时监听服务实例故障事件，一旦检测到实例失效，立即结合任务关键性等级与系统当前状态，智能生成最优恢复方案，最大限度降低故障对高优先级工作流的影响。

本节同样将问题建模为约束优化问题。设 W 为动态到达的分级工作流集合，研究目标是寻求一种可行的调度策略。为科学评估所提调度策略在“故障容错”与“成本控制”方面的综合效能，摒弃传统单一维度指标，创新性地提出“容错经济效能”作为核心评价标准：

$$f(j)=\begin{cases}1 & \text{任务流 } j \text{ 全部完成}\\ -1 & \text{任务流 } j \text{ 未完成}\end{cases} \tag{7}$$

$$FTEE=\frac{\sum_{j=1}^{j=|i|}\eta_{priority}*f(j)}{\sum_{j=1}^{j=|i|}cost_{i,j}^{k}} \tag{8}$$

其中，分子 $\sum\eta_{\text{priority}}\cdot f(j)$ 表征了经过调度后，系统中所有高优先级任务所获得的可靠性增益总和，它直接反映了系统在故障场景下维持核心服务的能力；分母 $\sum\text{cost}_{i,j}^{k}$ 则代表了达成此可靠性水平所消耗的总资源成本。

该指标的核心价值在于其综合性与导向性：它不追求不计代价的绝对可靠，而是致力于衡量每一单位成本投入所能换取的“可靠性收益”。一个高效的调度方案，应能在成本约束下，最大化此比率的值。这有力地驱动系统优化方向从单纯的“提升可靠性”转变为“以最优的成本效益比实现目标可靠性”，从而确保本研究在增强系统健壮性的同时，恪守了经济性原则，为解决工业界广泛面临的“冗余成本过高”问题提供了精准的量化依据与优化方向。

1.3. 关键技术

本课题的关键技术如下所述。

1.3.1. 研究点一：面向服务满意度的云服务分级工作流在线调度方法

图 1 展现了面向服务满意度的云服务分级工作流在线调度方法 RHWS 的调度架构，整体包括工作流初始化、任务调度以及反馈处理三部分。

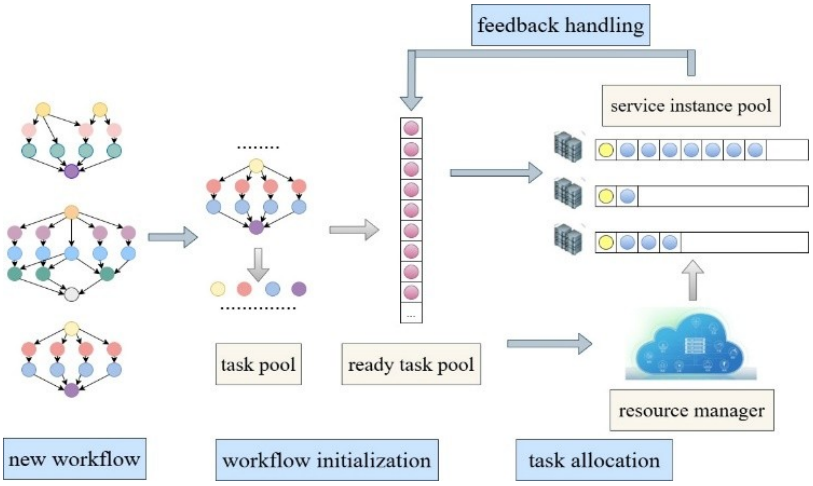


Fig. 1. 实时工作流调度框架

1.3.1.1. 问题模型

云计算平台面临的核心挑战之一在于处理用户动态提交的复杂工作流。这些工作流在提交前，其拓扑结构、到达时间与截止期限对云平台而言均为未知。为刻画此动态场景，本研究将云平台接收的工作流集合形式化定义为 $W=\{w_1,w_2,w_3,\ldots,w_n\}$，其中，每个工作流 $w_i \in W$ 可进一步表征为一个三元组 $w_i=\{a_i,ddl_i,G_i\}$，分别表示其到达时间、绝对截止时间及其拓扑结构。

工作流结构 Gi 被建模为一个有向无环图（DAG）$G_i=(T_i,E_i)$。$T_i=\{t_1,t_2,t_3,\ldots,t_{|Ti|}\}$是构成工作流 w_i 的子任务集合，其中每个元素 $ti_{,j}$ 代表工作流 w_i 中的第 j 个任务。E_i 为表征子任务间数据依赖关系的有向边集合。若子任务 $ti_{,p}$ 和 $ti_{,j}$ 之间存在数据依赖关系，则存在一条有向边 $e_{i,p,j}=(ti_{,p},ti_{,j}) \in E_i$。其权重 $w(e_{i,p,j})$表示从子任务 $ti_{,p}$ 到子任务 $ti_{,j}$ 传输的数据量。

由于工作流中的子任务可能依赖于多个前驱子任务或有多个后继子任务，我们定义 $pred(ti_{,j})$

与 succ($ti_{,j}$)分别为子任务 $ti_{,j}$ 的所有直接前驱与后继子任务集合。基于此依赖关系，子任务 $ti_{,j}$ 仅当其所有前驱子任务 pred($ti_{,j}$) 执行完毕并成功传输输出数据后，方可开始执行。这种数据依赖性要求云平台在调度任务时必须考虑子任务间的执行顺序，以确保数据的及时传递和子任务的顺利完成。

1.3.1.2. 算法设计

本研究将问题建模为一个优化服务满意度的问题，根据任务流的完成情况动态调整，激励任务的及时完成，并对未能按时完成的任务流施加惩罚。

用户提交新工作流时触发初始化流程。工作流预处理器基于每个未调度任务的到达时间、截止时间和 DAG 结构进行预处理，计算其最晚开始时间、最晚完成时间、最早开始时间、最早完成时间、集群最晚完成时间及子截止期。无直接前驱的任务被放入就绪任务池。

任务分配器按紧急性指标排序，并按序将就绪任务调度至合适的服务实例。若没有活跃或可租用实例满足条件，则选择能最早完成该任务的实例。

任务完成后启动反馈处理：更新任务池和就绪任务池，重新计算后继任务的最晚和最早时间，同时，资源管理器会与服务实例池进行协商，以决定服务实例的租用或释放。

阶段一：工作流初始化：任务关键路径与紧急性分析

工作流初始化的主要目标是确定到达工作流的内部子任务的调度顺序，涉及的关键参数包括预测最晚开始时间、最晚完成时间、最早开始时间、最早完成时间以及子截止期。该过程的伪代码如算法 1 所示。

Algorithm 1. Workflow initialization

Input: A workflow $w_i=(a_i,ddl_i,G_i,p_i)$

Output: Ready task pool Q_r, task pool Q_p, task parameters $\{est_{i,j}, ect_{i,j}, plst_{i,j}, plft_{i,j}, ddl_{i,j}, pcluft_{i,j}, E_{i,j}\}$

1: Compute $est_{i,j}, ect_{i,j}, plst_{i,j}, plft_{i,j}, ddl_{i,j}$ for all the tasks of workflow w_i

2: Compute cluster values and $pcluft_{i,j}$ for all the tasks of workflow w_i

3: Compute urgency $E_{i,j}$ using priority factor for all the tasks of workflow w_i

4: $Q_r \leftarrow \emptyset$, $Q_p \leftarrow \emptyset$

5: for each task $t_{i,j}$ in T_i do

6: if $t_{i,j}$ without immediate predecessors then

7: add $t_{i,j}$ to Q_r

8: else

9: add $t_{i,j}$ to Q_p

10: end if

11: end for

12: Return Q_r, Q_p and task parameters

新工作流到达时，需要计算每个子任务的预测最早开始时间 $\mathrm{est}_{i,j}$、最早完成时间 $\mathrm{ect}_{i,j}$、最晚开始时间 $\mathrm{plst}_{i,j}$ 和最晚完成时间 $\mathrm{plft}_{i,j}$。每个任务的子截止期设置为其预测最晚完成时间。

其中，各个子任务的预测最晚开始时间（$plst_{i,j}$）与预测最晚完成时间（$plft_{i,j}$）采用逆向递推计算，以确定任务调度的关键路径与时序约束边界。

$$plst_{i,j}=\begin{cases} ddl_i - pet_{i,j}^{k} - dt_{vm} & when\ \ \mathrm{succ}(t_{i,j})=\phi \\ min_{t_{i,s}\epsilon succ(t_{i,j})}\left\{plst_{i,s} - pt_{j,s}^{i,\beta}\right\} - pet_{i,j}^{k} & else \end{cases} \tag{9}$$

$$plft_{i,j}=plst_{i,j}+pet_{i,j}^{k} \tag{10}$$

其中，$pet_{i,j}^{k}$是子任务$t_{i,j}$在服务实例 k 上的预测执行时间，dt_{vm}为服务实例的启动延时，$pt_{j,s}^{i,\beta}$代表从子任务$t_{i,j}$中把数据传输给子任务$t_{i,s}$所需要的传输时间。

为精确量化就绪任务池中任务的调度顺序，本研究设计了任务紧急性指标（$\mathrm{E}_{i,j}$），其计算包含三个步骤：

1. 计算聚簇值$\mathrm{cluster}(\mathrm{t}_{i,j})$：采用向上排序法，确定子任务在工作流拓扑中的层级

$$\mathrm{cluster}(\mathrm{t}_{i,j})=\mathrm{max}_{\mathrm{t}_{i,s}\epsilon \mathrm{Pre}(\mathrm{t}_{i,j})}\mathrm{cluster}(\mathrm{t}_{i,s})+1 \tag{11}$$

2. 聚簇最晚完成时间 $\mathrm{pcluft}_{i,j}$：定义同一聚簇内子任务的最晚完成时间边界，在保证

后续任务开始时间不受影响的前提下，使各任务的执行时间更具灵活性。

$$pcluft_{i,j} = max_{t_{i,s} \epsilon T_i, cluster(t_{i,s}) = cluster(t_{i,j})} plft_{i,s} \tag{12}$$

3. 紧急性指标 $E_{i,j}$: 综合任务优先级、时间余量与预计开始时间，形成最终的调度先后排序依据。此处$\alpha_{priority}$为与任务优先级相关的参数。该紧急度计算机制在确保不违反截止时间的前提下，实现了对工作流优先级的综合考虑。

$$E_{i,j} = \alpha_{priority} * (pcluft_{i,j} - est_{i,j}) + est_{i,j} \tag{13}$$

阶段二：任务调度分配

在任务分配阶段，就绪任务池中的任务按照紧急度指标升序排列。如算法 2 所示，任务随后被依次调度到服务实例上。任务将被分配给在满足子截止期约束前提下（第 4-15 行）能实现服务选择效益最大化的服务实例。若当前无活跃实例满足约束条件，则在资源限额允许范围内租用新实例（第 17-28 行）。当活跃实例与可租用实例均无法满足约束时，则选择能提供最早完成时间的实例。分配给空闲实例的任务在接收完前驱任务数据后立即启动，而分配给繁忙实例的任务则需在队列中等待（第 29-37 行）。

Algorithm 2. Task Allocation
Input: Ready task pool Q_r, active service instances Vm_{active}, rentable service instances

$Vm_{rentable}$, resource constraints

Output: The task in Q_r is scheduled to its target service instance

1: Sort tasks in Q_r by urgency $E_{i,j}$ in ascending order

2: for each task $t_{i,j}$ in Q_r do

13: vm_{target} =$\emptyset$,maxServiceChoice←-∞, vm_{fast} = $\emptyset$, fastPft ← +∞

14: for each service instance vm_k in Vm_{active} do

15: Compute $pst_{i,j}^k$, $pft_{i,j}^k$

16: Compute $serviceChoice(t_{i,j}^k)$ using priority-weighted cost and time savings

17: if $serviceChoice(t_{i,j}^k)$ > maxServiceChoice and $pft_{i,j}^k \leq ddl_i$ then

18: maxServiceChoice ← $serviceChoice(t_{i,j}^k)$

19: vmtarget ← vm_k

20: end if

21: if fastPft > $pft_{i,j}^k$ then

22: fastPft ← $pft_{I,j}^k$

23: vmfast ← vmk

24: end if

25: end for

26: vm_{active} =$\emptyset$, maxActiveServiceChoice ← -∞, $vm_{activeFast}$ = $\emptyset$, activeFastPft ← +∞

27: for each service instance vm_k in $Vm_{rentable}$ do

28: Compute $pst_{i,j}^k$, $pft_{i,j}^k$

29: Compute$serviceChoice(t_{i,j}^k)$ using priority-weighted cost and time savings

30: if $serviceChoice\left(t_{i,j}^k\right)$> maxActiveServiceChoice and $pft_{i,j}^k \leq ddl_{i,j}$ then

31: maxActiveServiceChoice ← $serviceChoice\left(t_{I,j}^k\right)$

32: vm_{active} ← vm_k

33: end if
34: if activeFastPft > $pft_{i,j}^{k}$ then
35: activeFastPft ← $pft_{I,j}^{k}$
36: $vm_{activeFast}$ ← vm_{k}
37: end if
38: end for
39: if maxServiceChoice < maxActiveServiceChoice && $vm_{target} \neq \emptyset$
40: lease a new service instance with type vm_{active} and schedule $t_{i,j}$ on it
41: else if maxServiceChoice > maxActiveServiceChoice &$vm_{target} \neq \emptyset$
42: schedule $t_{i,j}$ on vm_{target}
43: else if vm_{target} =∅&vm_{active}=∅ &$vm_{activeFast}$≠∅
44: lease a new service instance with type $vm_{activeFast}$ and schedule $t_{i,j}$ on it
45: else if
46: schedule $t_{i,j}$ on vm_{fast}
47: end if
48: end for

该阶段的目标是为每个任务找到合适的服务实例，在满足执行约束的同时最大化服务选择效益。在为任务分配合适的服务实例时，需权衡时间成本与资源成本，并依据任务优先级进行动态调整。为此，本研究提出了服务选择收益指标（$SelectEarning_{i,j}^{k}$）。

$$Cost_{max_{i,j}} = max_{k=1}^{|\mu|} \{ P(\mu)^{k} * pet_{i,j}^{k} \} \quad (14)$$

$$SelectEarning_{i,j}^{k} = (1 - \gamma_{priority}) * \frac{ddl_i - pft_{i,j}^{k}}{ddl_i - T_{start_i}} + \gamma_{priority} * \frac{Cost_{max_{i,j}} - P(\mu)^{k} * pet_{i,j}^{k}}{Cost_{max_{i,j}}} \quad (15)$$

其中，$\gamma_{priority}$为与优先级相关的权重参数，用于动态调节时间效益与成本节约在决策中的比重。

阶段三：反馈处理

反馈处理流程在任务完成时触发，如算法 3 所示。首先，同一服务实例上的等待任务在其所有前驱任务数据接收完成后立即开始执行（第 1-3 行）。对于每个直接后继任务，若其所有前驱任务均已被调度且至少有一个已完成，则该任务进入就绪状态并被移入就绪任务池等待调度（第 4-8 行）。

Algorithm 3. Feedback Processing

Input: Completed task $ti_{,j}$, service instance vm_k, workflow w_i, task pool Q_p

Output: Updated ready task pool Q_r

1: if there exits waiting task on vmk then

3: The first waiting task in the waiting queue begins execution once it receives the data transmitted by its predecessor tasks.

49: end if

50: for each successor task $t_{i,s} \in Succ(t_{i,j})$ do

51: if all predecessor tasks of $t_{i,s}$ are scheduled and at least one has finished execution then

52: Move $t_{i,s}$ to the ready task pool Q_r, and remove from task pool Q_p

53: end if

54: end for

1.3.1.3. 实验验证与分析

我们基于 Java 实现了离散事件模拟器，并将其部署在配备 2.5GHz Intel Core i9 处理器和

32GB 内存的计算机上。实验采用的云服务平台支持四种可动态租用的服务实例类型，租用周期设置为 60 分钟。平台的总 CPU 和内存资源受限，从而限制了动态租用实例的总资源消耗量。

在工作流模板方面，我们采用了 15 个具有五种不同结构（CyberShake、Inspiral、Montage、Epigenomics 和 Sipht）和三种不同任务规模的现实工作流应用。这些模板涵盖了工作流的所有基本结构特征，包括其组件和组合方式。工作流优先级被随机分配，以模拟真实的多工作流调度场景。工作流到达时间：工作流随机到达云服务平台。我们从 15 种工作流中随机选择，并使用 λ=0.1 的泊松分布生成到达间隔时间。

1.3.1.3.1. 消融实验

图 2 和图 3 展示了在 SWMDSA 算法上进行的消融实验。RHWS 算法是 SWMDSA 的增强版本，其中排序和调度模块分别进行了改进。图 2 评估了 SWMDSA 中排序模块的影响。实验结果表明，虽然新的排序模块对服务满意度有所提升，但改进幅度有限。这主要是因为排序仅在任务就绪后进行，且就绪任务池的规模相对较小。此外，新的排序方法在每个簇内优先处理临近截止时间的任务，这限制了排序模块进一步的改进空间。图 3 展示了 SWMDSA 调度模块的消融实验。结果表明，新的调度模块使得服务满意度指标显著提升，表现出明显的性能改进。然而，该服

务满意度仍低于 RWPS 调度算法所达到的指标。这表明，尽管优化调度模块能带来实质性改进，

但排序模块和调度模块之间存在互补效应，二者的协同增强才能带来更显著的总体性能提升。

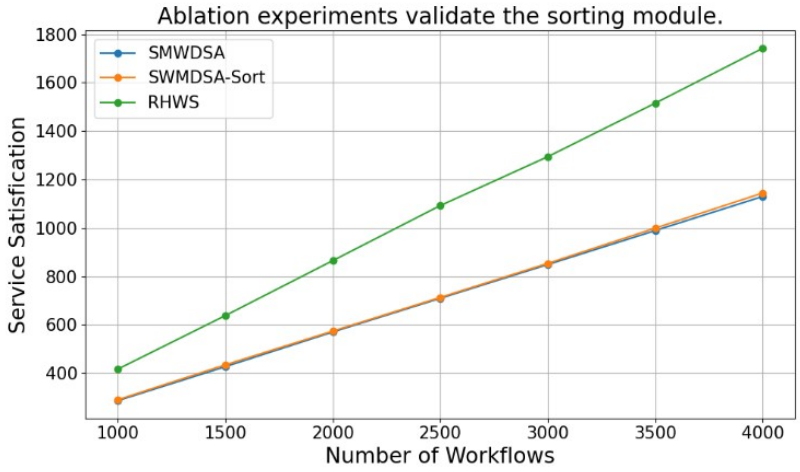


Fig. 2. 排序模块的消融实验

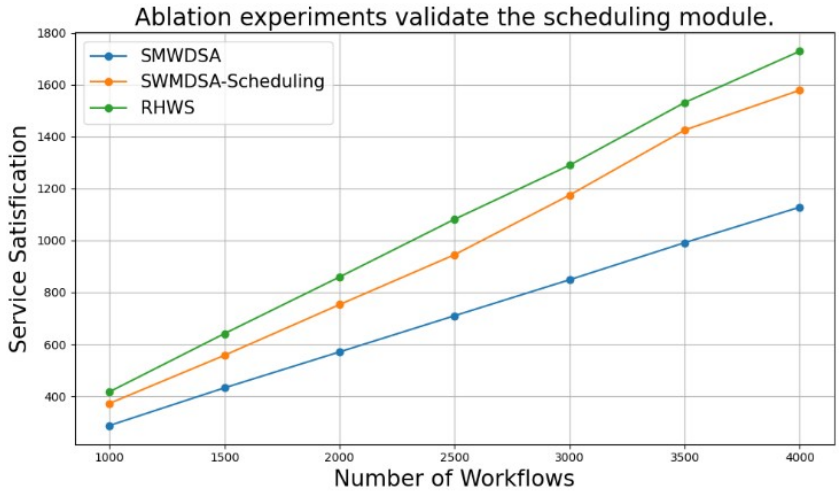


Fig. 3. 调度模块的消融实验

1.3.1.3.2. 对比实验

为评估本研究所提出的 RHWS 调度算法的性能，我们将其与三种先进的动态工作流调度算

法——ROSA、SMWDSA 和 NOSF 进行了全面对比实验。实验结果表明：

- 一致性优势：在所有测试场景下（包括不同截止时间系数、工作流规模、执行时间方差及传输时间方差），RHWS 算法均取得了最高的服务满意度。

- 性能提升显著：相较于 ROSA 和 SMWDSA 算法，RHWS 将服务满意度平均提升了 60%以上；与性能最佳的基线算法 NOSF 相比，仍有超过 20%的性能提升。



Fig. 4. 执行时间方差对算法性能的影响

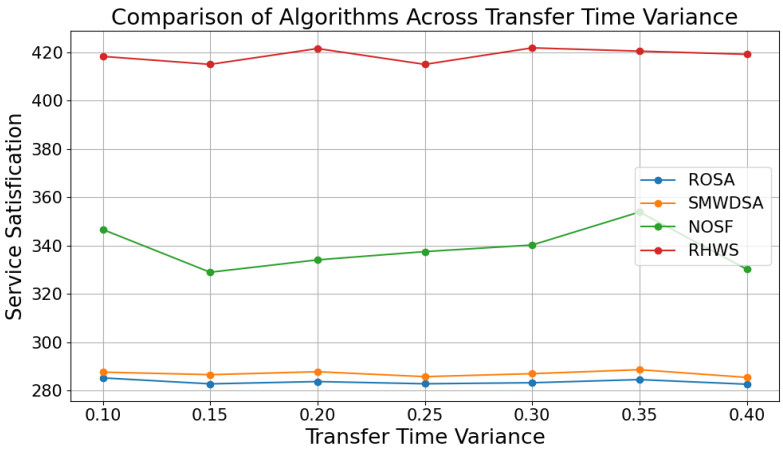


Fig. 5. 传输时间方差对算法性能的影响

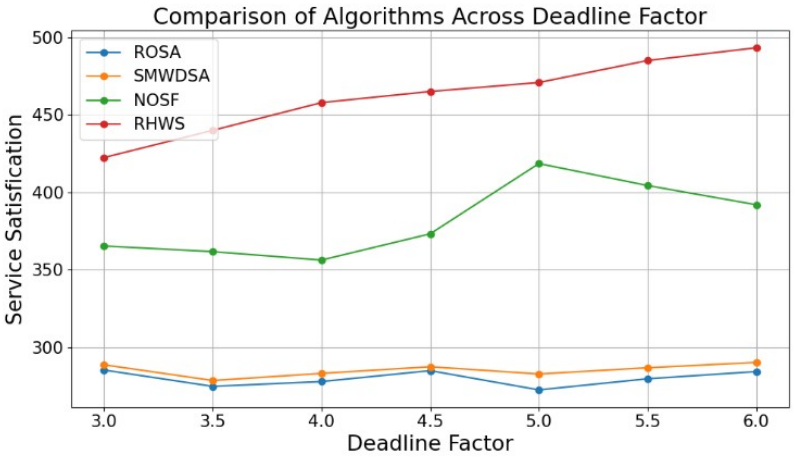


Fig. 6. 截止时间系数对算法性能的影响

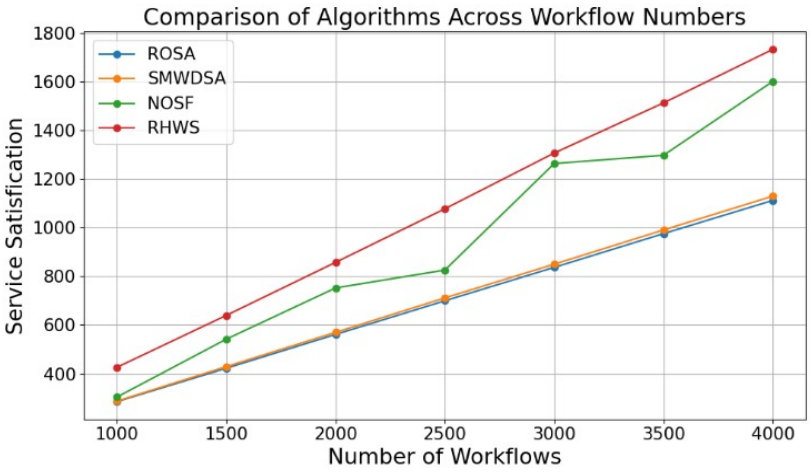


Fig. 7. 工作流数量对算法性能的影响

1.3.2. 研究点二：面向可靠性的云服务分级工作流在线调度方法

1.3.2.1. 问题模型

在云计算环境中，用户频繁提交复杂任务流至云平台进行处理。此类任务流在提交前，其到达时间、截止时间及内部结构均为未知。任务流可建模为集合 $W=\{w_1,w_2,w_3,\ldots,w_n\}$，其中每个工作流 w_i 可进一步表示为三元组 $w_i=\{a_i,ddl_i,G_i\}$，分别表示到达时间、截止时间及其结构。

工作流结构 Gi 采用有向无环图（DAG）表示，即 $G_i=(T_i,E_i)$。其中，$T_i=\{t_1,t_2,t_3,\ldots,t_{|Ti|}\}$是构成工作流 w_i 的子任务集合，其中每个元素 $ti_{,j}$ 代表工作流 w_i 中的第 j 个任务。E_i 是 T_i 中子任务之间的数据依赖的有向边集合，如果任务 $ti_{,p}$ 和 $ti_{,j}$ 之间存在数据依赖关系，那么在 E_i 中存在一条直接边 $e_{i,p,j}$，表示为（$ti_{,p}$，$ti_{,j}$），其权重 $w(e_{i,p,j})$表示需传输的数据量。

由于工作流中的子任务可能依赖于多个前驱子任务或有多个后继子任务，我们定义 $pred(ti_{,j})$ 与 $succ(ti_{,j})$ 分别表示子任务 $ti_{,j}$ 的所有直接前继与后继子任务集合。由于工作流子任务之间的数据依赖性，子任务 $ti_{,j}$ 只有在收到其所有前驱任务 $pred(ti_{,j})$的输出数据后，才能开始执行。这种数据依赖性要求云平台在调度任务时必须考虑子任务间的执行顺序，以确保数据的及时传递和子任务的顺利完成。

1.3.2.2. 调度策略

- 新任务到达

系统首先对任务池与就绪任务池中的子任务进行优先级重排序，确保其执行顺序符合工作流逻辑约束。随后，依据排序结果将就绪子任务分派至合适的服务实例，并动态更新任务池中满足就绪条件的子任务。

- 子任务完成时

服务实例完成当前子任务后，立即检查是否存在待执行的子任务。若有，则启动下一任务；若无，则标记该实例为空闲状态。同时，系统持续监测任务池，将新满足就绪条件的子任务移入就绪任务池，保障调度连续性。

- 服务实例出现故障

当检测到服务实例故障时，系统立即将其上正在执行、等待执行及所属任务流中后续已分配子任务全部重置为就绪状态，回置至就绪任务池，触发任务重分配与系统重配置。

1.3.2.3. 计算模型

将问题建模为系统可靠性优化问题，通过调度策略满足不同优先级任务流的可靠性要求，并在服务实例故障时对已分配子任务进行重配置。

算法流程如下：

用户提交新工作流时触发初始化流程，无直接前驱的任务放入就绪任务池。任务分配器按任务最晚完成时间排序，将就绪任务调度至合适服务实例。在任务调度的时候，会根据就绪任务池中的任务规模情况采用强化学习调度器或强化学习与模拟退火的混合优化调度器。算法实

时计算可靠性感知的动态可靠因子的分界值，并在调度的过程中根据分界值进行优先级自适应分配，使得高优先级的任务分配到稳定性高的服务实例，反之亦然。除此之外，在调度过程中采用多目标优化，综合考虑到执行成本、可靠性以及时间紧迫性几个指标。

与此同时，算法实时监听服务实例故障的情况。在检测到服务实例失效时，结合任务优先级与系统实时状态，生成最优恢复方案，最大限度降低故障对高优先级工作流的影响。

阶段一：计算可靠性感知的动态可靠因子的分界值

算法实时计算动态可靠因子分界值，用于划分不同优先级任务可分配的服务实例稳定性范围，避免低优先级任务“饿死”，提高实例使用率。

为量化服务实例的可靠性，本研究为每个服务实例 j 引入了稳定性因子 $\zeta(j)$。该因子是一个归一化指标，取值范围为 $[0, 1]$，综合考虑了服务出现故障的频率和从故障中恢复的速度，反映了实例在面对故障时保持连续服务的能力。其中，$\zeta(j) = 0$ 表示实例完全不可用，$\zeta(j) = 1$ 则表示实例绝对可靠。稳定性因子直接决定了任务被调度至该实例时的预期成功率。

为评估系统整体服务实例的稳定性分布，我们计算所有已启动服务实例稳定性因子的平均值与方差，如公式 (16) 与 (17) 所示

$$\mu=\frac{1}{|SI|}\sum_{j=1}^{|SI|}\zeta(j) \tag{16}$$

$$\sigma^2=\frac{1}{|SI|}*\sum_{j=1}^{|SI|}(\zeta(j)-\mu)^2 \tag{17}$$

为实现任务流的动态分级，我们首先计算高优先级任务流的基准横坐标值 x。该值由高优先级任务流的子任务总数与系统内所有任务流子任务总数的比值决定，反映了高优先级任务在系统中的比重，是确定分级阈值的关键依据，具体计算如公式 (19) 所示。基于该比值，我们通过高斯分布函数动态计算分级因子的分界值 θ，如公式 (20) 所示。此分界值用于精确区分高优先级与低优先级任务流。

$$f(i)=\begin{cases}1 & \textit{任务流 } i \text{ 是高优先级任务流}\\ 0 & \textit{任务流 } i \textit{ 是低优先级任务流}\end{cases} \tag{18}$$

$$x=\mu+\frac{\sum_{j=1}^{|i|}\sum_{k=1}^{|S|_i}f(i)}{2*\sum_{j=1}^{|i|}\sum_{k=1}^{|S|_i}1} \tag{19}$$

$$\theta=\frac{1}{\sqrt{2\pi\sigma^2}}e^{-\frac{(x-\mu)^2}{2\sigma^2}} \tag{20}$$

此外，本研究引入强化学习机制，通过分析历史任务完成情况与实时系统负载状态，对可

靠性阈值进行动态调整。这种方法克服了静态阈值的局限性，实现了更精细化、自适应性的任务分级，从而进一步提升系统资源分配的效率与整体可靠性。

阶段二：任务调度分配策略

在本研究的任务分配阶段，我们根据当前时刻就绪任务集的规模进行多样化调度，具体如图八所示。对于小规模任务采用强化学习调度器完成调度。而对于大规模任务，采用强化学习和模拟退火的混合优化调度器。

针对强化学习调度器，我们设计了一种融合动态可靠因子分界值的状态增强型强化学习智能体，用以智能指导任务至服务实例的映射决策。该智能体的状态表示得到了显著增强，其状态向量综合集成八个关键维度，并显式地嵌入了前一阶段计算得出的动态阈值信息，从而极大地提升了策略的感知泛化能力。为避免无效探索并加速收敛，我们在探索阶段引入了动作过滤机制，确保智能体仅能从满足预设可靠性约束的合法动作中进行选择。在此基础上，算法采用 Q-learning 作为学习核心，通过 ϵ-贪婪策略平衡探索与利用，并持续更新 Q 表以追求长期累积奖励的最大化，最终实现高效且可靠的任务调度。

为克服单一优化算法易于陷入局部最优的局限，本研究还设计了一种模拟退火与强化学习

的混合优化调度器。该调度器首先利用强化学习智能体生成高质量的初始任务分配方案，以此作为模拟退火优化器的起点，有效跳过了传统模拟退火从随机解开始的缓慢搜索阶段。在模拟退火的迭代优化过程中，我们创新性地改进了其解接受准则，引入了“可靠性增益因子”对接受概率进行加权调整，确保算法在追求调度性能提升的同时，能够充分兼顾方案的稳定性与可靠性。此外，算法还内置了有效的容错回退机制：若混合优化因极端条件未能产生更优解，系统将自动回退至纯强化学习策略的输出，从而保障了调度系统在各种场景下的鲁棒性与可用性。

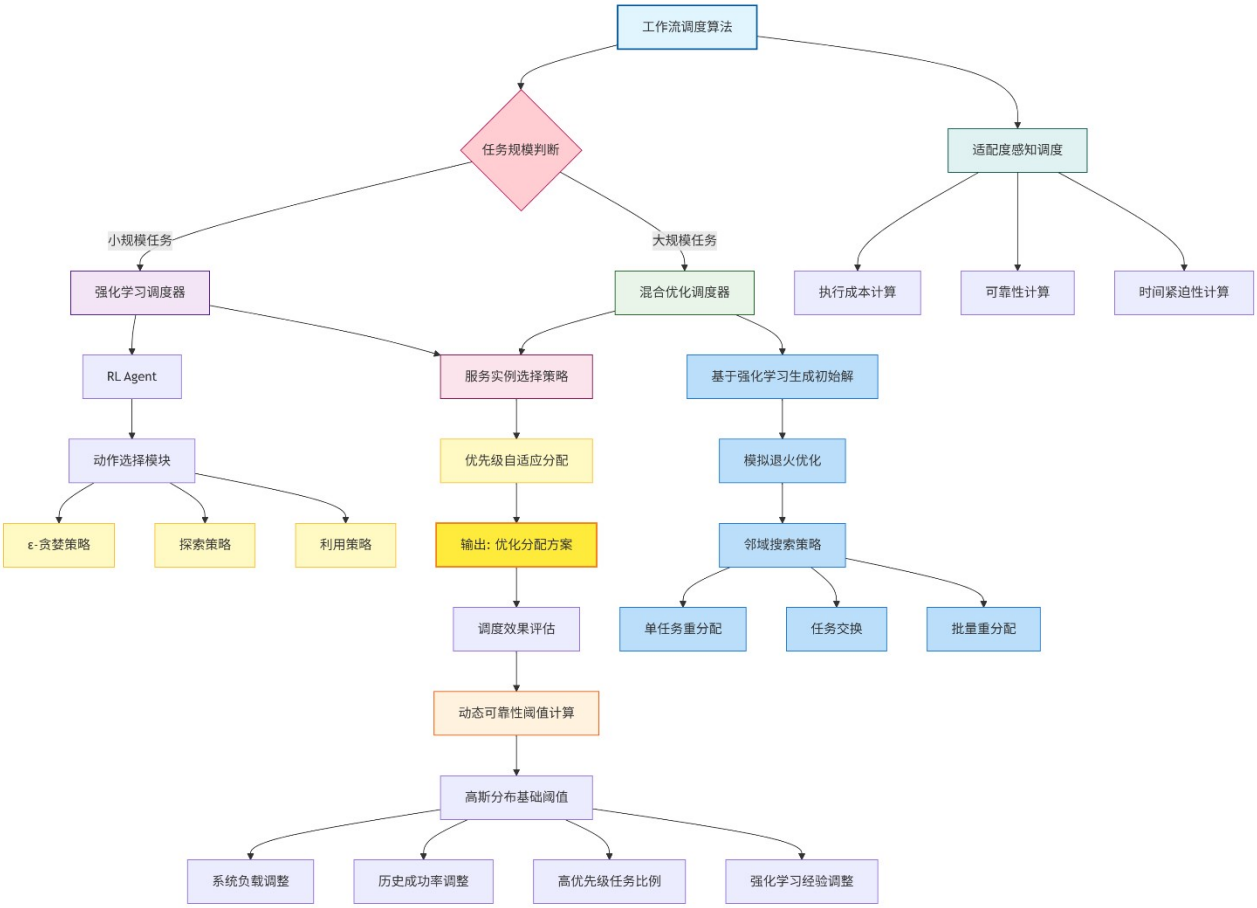


Fig. 8. RHOTS 算法调度流程

为支撑精细化的调度决策，本研究还构建了一个多因子适配度函数，该函数综合权衡了执行成本、可靠性与时间约束三大关键指标。具体而言，执行成本严格遵循服务实例计费模型，根据任务的实际运行时长进行量化；可靠性维度则引入了动态惩罚机制，当服务实例稳定性因子偏离系统计算出的动态阈值时，将施加相应的惩罚项，以引导调度器优先选择更可靠的服务实例；时间紧迫性模拟任务的紧急程度，其值随任务完成时间逼近截止时间而增加，归一化至 [0, 1] 区间。越接近 1，表示时间压力越大。该适配度函数作为调度算法的核心指标，确保了最终方案能够在多个优化目标间取得最佳平衡. 其中，$\gamma_{priority}$为与优先级相关的权重参数

$$Execution_{i,j}^{k} = \gamma_{priority} * pet_{i,j}^{k} * vmPrice \tag{21}$$

$$\text{约束条件：} \begin{cases} \zeta(j) \geq \theta & \text{若任务为高优先级} \\ \zeta(j) < \theta & \textit{若任务为低优先级} \end{cases} \tag{22}$$

$$Reliability^{k} = \begin{cases} \alpha * |\zeta(j) - \theta| & \textit{若违反上述约束} \\ 0 & \text{否则} \end{cases} \tag{23}$$

$$Ergency_{i,j}^{k} = \frac{pft_{i,j}^{k} - T_{start_i}}{ddl_i - T_{start_i}} \tag{24}$$

$$Fitness_{i,j}^{k} = aExecution_{i,j}^{k} + bReliability^{k} + cErgency_{i,j}^{k} \tag{25}$$

阶段三：服务实例故障下的重配置

算法实时监听是否出现服务实例故障的情况。在检测到服务实例失效时，将实例上的所有任务重新投入就绪任务池。在下一轮调度中，结合任务优先级情况与系统实时状态，这些任务将优先被分配至空闲或新申请的、满足可靠性要求的服务实例上，最大限度降低故障对高优先级工作流的影响。

1.3.2.4. 实验验证与分析

我们基于 Java 实现了离散事件模拟器，并将其部署在配备 2.5GHz Intel Core i9 处理器和 32GB 内存的计算机上。实验采用的云服务平台支持四种可动态租用的服务实例类型，租用周期设置为 60 分钟。平台的总 CPU 和内存资源受限，从而限制了动态租用实例的总资源消耗量。

在工作流模板方面，我们采用了 15 个具有五种不同结构（CyberShake、Inspiral、Montage、Epigenomics 和 Sipht）和三种不同任务规模的现实工作流应用。这些模板涵盖了工作流的所有基本结构特征，包括其组件和组合方式。工作流优先级被随机分配，以模拟真实的多工作流调度场景。工作流到达时间：工作流随机到达云服务平台。我们从 15 种工作流中随机选择，并使用 λ=0.1 的泊松分布生成到达间隔时间。

为评估本研究提出的调度算法性能，我们针对两种场景设计了对比实验：不同工作流数量和不同系统失败概率。在每种场景下，本 RHOTS 算法均与两种先进的动态工作流调度算法——基于重配置的 ROSA 和基于冗余的 DEFT[17]进行了性能比较。

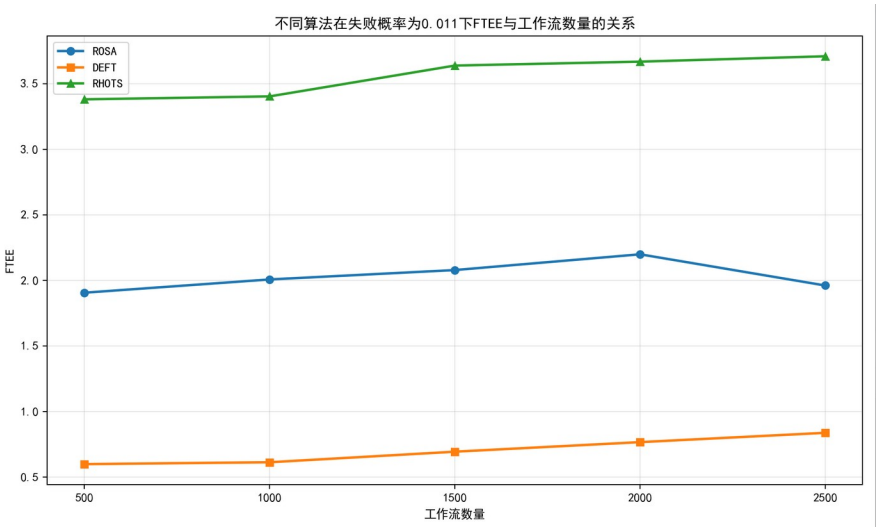


Fig. 9. 失败概率对算法性能的影响

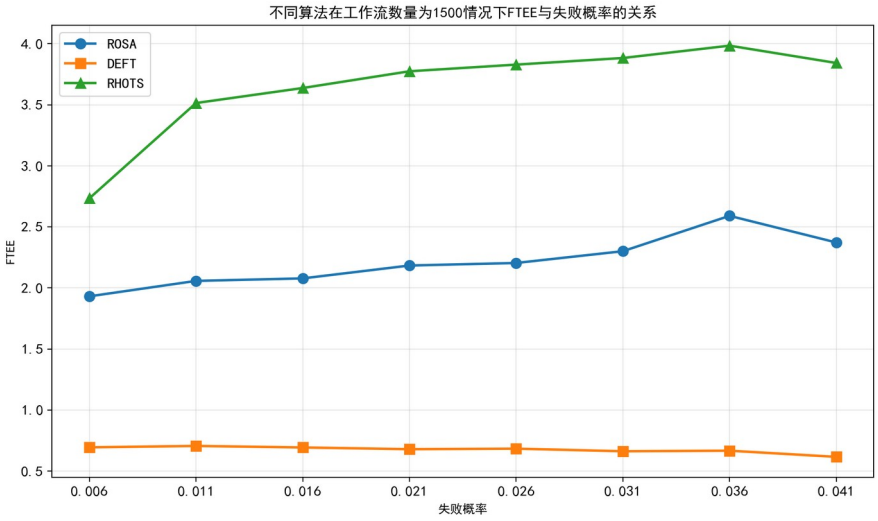


Fig. 10. 工作流数量对算法性能的影响

实验结果表明：本文提出的 RHOTS 算法在容错经济效能（FTEE）方面展现出显著优势。在不同失败概率和工作流数量两种测试场景下，RHOTS 的 FTEE 值均稳定保持在 3.38-3.98 的较高水

平，相比 ROSA 算法性能提升约 70%以上，相比 DEFT 算法提升超过 300%。结果表明，RHOTS 算法不仅具有优异的故障容忍能力，还能适应不同规模的工作流负载。

1.4. 论文计划

时间	研究内容
2024.11-2024.12	开题报告
2024.12-2025.01	调研任务调度相关技术
2025.02-2025.05	研究点一的算法方案设计和仿真实验
2025.06-2025.09	研究点二的算法方案设计和仿真实验
2025.10-2025.11	算法对比和优化
2025.12-2026.03	论文撰写和修改
2026.04-2026.05	根据导师和外审意见修订论文
2026.05	准备答辩

1.5. 论文进度及目标

截至目前，项目已按计划完成以下工作：系统开展了云服务任务调度与强化学习相关技术的理论调研，深入分析了当前云服务环境下任务调度面临的核心挑战，并在此基础上完成了实时分级工作流在线调度的整体架构设计。

在算法实现方面，项目围绕两个核心研究方向分别取得了实质性进展：

第一，实现了面向服务满意度的实时分级工作流调度算法。该算法创新性地采用向上秩排序方法确定任务执行顺序，并引入自适应优先级调度机制，实现了对不同优先级任务的差异化成本优化——低优先级任务侧重资源成本控制，高优先级任务保障时间效率。该设计显著提升了系统整体服务满意度，相关研究成果已在 EI 会议上发表。

第二，构建了面向可靠性的实时分级工作流调度算法。该算法通过基于强化学习的容错决策机制，实现了可靠性感知的动态阈值调整，并采用混合优化算法完成资源分配，有效增强了系统在不确定环境下的鲁棒性。围绕该创新成果的发明专利已完成提交，目前处于待受理阶段。

相关研究工作已准备中期答辩。

二、论文进展情况

2024.11-2025.01 已完成

调研任务调度相关技术，分析业务需求

2025.02-2025.03 已完成

完成了首个研究点“面向服务满意度的云服务分级工作流在线调度方法”的核心算法设计，并完成了对比实验的文献调研与算法选型。

2025.03-2025.04 已完成

通过复现基线算法并改造实验环境，实现了自研的 RWPS 算法，并围绕服务满意度等核心指标展开了初步评估。

2025.04-2025.05 已完成

深入开展了多维度对比实验，通过复现多种经典算法并从服务满意度、成本等角度进行系统对比，进一步验证了本研究提出算法的综合性能优势。

2025.05-2025.06 已完成

完成了第二个研究点“面向可靠性的云服务分级工作流在线调度方法”的核心算法设计，并完成了对比实验的文献调研与算法选型。

2025.06-2025.08 已完成

根据设计文档，搭建实现了自研的基于强化学习的动态可靠性调度算法

2025.09-2025.10 已完成

复现经典对比算法进行验证

2025.10-2025.12 已完成

对已完成的任务调度算法进行调优，编写相关学术论文，准备中期答辩

三、工作成果

3.1 已完成的工作成果

完成了对业内依赖性工作流的实时调度解决方案的调研，分析了支持成本和可靠性问题的解决方案。学习了启发式算法、强化学习方法的底层逻辑、基础架构和应用方法。掌握了现有任务调度方法的设计思路和整体流程。

完成了基于多优先级实时依赖性工作流的任务调度方法的技术选型，设计了动态调度框架。本课题采用启发式算法，提出采用向上秩排序的方式，以聚簇为单位对子任务排序并辅以根据工作流的优先级情况动态调整调度策略，从而实现对多优先级依赖性工作流的基于服务满意度的在线调度方法；除此之外，本课题还采用了强化学习与模拟退火的混合优化实现自适应调度，并辅以重配置机制用于抵御服务实例故障的场景，从而实现对多优先级依赖性工作流的基于可靠性的在线调度方法。

3.2 取得的阶段性成果

1) 调研业内现有的相关算法，通过分析总结系统需求

对业界主流的针对成本和可靠能力进行了全面系统的调研。通过对这些文献阅读，总结出云环境下的在线任务调度的需求和痛点，形成了详细的需求文档。

2) 研究基于面向服务满意度的云服务分级工作流在线调度方法

本课题提出了面向服务满意度的云服务分级工作流在线调度方法。该方法通过优先级驱动的子截止期限分配、成本效益自适应的服务选择与基于反馈的实时调整机制三者协同，系统性解决了在满足差异化截止期限约束的同时，优化资源效率与应对运行时不确定性的关键问题。

基于本课题的理论研究部分，于 IEEE CCSB 2025 会议上发表论文《Uncertainty-Aware Scheduling of Hierarchical Realtime Workflows with Deadline Constraints in Cloud Environments》[27]并作口头演讲。

3) 研究基于面向可靠性的云服务分级工作流在线调度方法

本课题提出了面向可靠性的云服务分级工作流在线调度方法。该方法通过动态可靠性分界值机制，自适应调整可靠性要求，结合强化学习智能体进行多目标优化决策，并采用模拟退火算法进行局部优化，实现了在保证高优先级任务可靠性的同时，提高系统整体性能。

基于本课题理论研究部分，已完成专利申请《面向可靠性的云服务分级工作流在线调度方法及装置》的撰写，并已提交专利代理公司受理。

3.3 课题的创新之处

本课题的创新点主要有：

研究点一：提出面向服务满意度的云服务分级工作流在线调度方法

1) 基于向上秩分簇的静态任务排序策略

针对云环境中具有复杂依赖关系的工作流任务调度问题，本研究提出了一种基于向上秩的分簇预排序机制。该策略通过计算每个任务的向上秩值，构建任务的拓扑执行顺序，并将其划分至不同

的执行簇中。每个簇内的任务具有相似的执行层级与依赖关系，从而在预处理阶段即可明确任务的关键路径与执行优先级。相较于传统的动态优先级调度方法，该静态排序策略不仅降低调度过程中的实时计算开销，还通过“预测簇最晚完成时间”这一指标，增强了对任务执行时间窗口的柔性控制，在保障整体截止时间的前提下，提高了调度系统的可预测性与稳定性。

2) 面向多目标优化的自定义启发式分配算法

在任务实例分配阶段，本研究设计了一种轻量级启发式调度算法，可在多目标之间实现有效权衡。该算法通过构建“服务选择效益”函数，将时间成本与资源成本统一于同一评估框架，并引入优先级权重因子实现任务差异化调度。高优先级任务倾向于分配至高性能服务实例以缩短执行时间，而低优先级任务则更注重资源成本的最小化。该机制在保证高优先级任务时限要求的同时，显著降低了低优先级任务的租赁开销，从而在整体上实现了服务满意度的优化。

3) 服务满意度驱动的闭环反馈机制

为应对云环境中任务执行时间与数据传输时间的不确定性，系统设计了基于任务完成状态的反馈调节机制。该机制在任务执行完毕后动态更新后续任务的状态，重新评估其紧急度与可调度性。同时，系统引入服务满意度函数，对超时任务实施基于优先级的梯度惩罚，从而在系统层面形成闭环优化。

研究点二：提出面向可靠性的云服务分级工作流在线调度方法

1) 动态自适应的可靠性阈值机制

针对传统静态阈值难以反映系统实时状态的问题，本研究提出一种融合统计学习与强化学习的动态阈值生成方法。该方法首先依据服务实例稳定性因子的分布特征，构建高斯概率密度函数以确

立基础阈值；进而引入系统负载感知模块，实现阈值与系统运行状态的联动调整；最终通过强化学习智能体，基于历史调度效果对阈值进行在线微调，形成一套具备统计合理性、负载敏感性及演化自适应能力的智能决策体系，显著提升了任务分级的准确性与环境适应性。

2) 深度混合的强化学习与模拟退火优化框架

为克服单一算法在求解质量与效率上的局限，本研究构建了强化学习与模拟退火深度融合的混合优化架构。设计涵盖任务属性、系统状态与资源状况的八维状态空间，确保智能体对环境具备全面感知能力；建立融合任务完成、时间效率、成本控制与可靠性匹配的多目标奖励函数，精准引导策略优化。在调度过程中，采用“RL 生成初始解—SA 精细搜索”的两阶段策略，既利用强化学习的快速响应能力保障实时性，又借助模拟退火在解空间的强大探索能力实现全局优化，有效平衡了求解速度与调度质量。

3) 多目标可靠性均衡机制

为实现可靠性保障与经济效益的协同优化，本研究设计了一种成本与可靠性平衡模型。通过构建综合执行成本、时间紧迫性与可靠性惩罚的适应度函数，引导调度系统在满足可靠性约束的前提下实现综合性能最优。提出基于偏离度的动态惩罚策略，对不满足动态可靠性分界值约束的任务实施梯度惩罚，并针对低可靠性服务实例引入风险评估成本，从经济维度强化系统的可靠性导向。进一步地，根据任务优先级与类型特征，自适应调整优化目标的权重分配，形成差异化的资源调度策略，从而在复杂云环境中实现可靠性、时间效率与资源成本的高水平协同。

四、计划及进度安排

第一阶段：论文核心研究深化与初稿撰写（2025 年 12 月 – 2026 年 2 月）

1) 核心任务： 完成毕业论文初稿的系统性撰写。

2) 研究内容补充完善：

a) 补充增加并设计基于强化学习的可靠性对比算法，进行充分的实验测试。

b) 利用上述算法的测试结果，迭代优化并验证论文核心的“容错经济效能”指标。

c) 对现有算法模型进行精炼与完善，提升其性能与论证力度。

第二阶段：论文修订与审阅意见反馈（2026 年 2 月 – 2026 年 5 月）

1) 核心任务： 根据内、外部审阅意见对论文进行全面修订与提升。

2) 具体工作：

a) 充分吸纳导师的指导意见，进行多轮修改。

b) 针对同行专家或外审的评审意见，对论文的结构、内容、论证过程及格式进行细致修订与完善。

第三阶段：答辩准备与最终成果提交（2026 年 5 月）

- 核心任务： 完成所有结题答辩准备工作。
- 具体工作：
  - 整理最终版毕业论文及相关成果材料。
  - 准备答辩陈述、演示文稿，并进行预演练习。
  - 参加结题答辩。

五、问题及整改方案

基于面向可靠性的云服务分级工作流在线调度方法的对比算法进一步完善

目前，实验部分已纳入了业界两种常用的稳健性调度算法作为对比基准，并完成了相关的对比测试，这初步验证了本方法相对于传统方案的性能优势。

然而，当前的对比实验体系在技术维度的覆盖上仍有不足。具体而言，尚缺乏与采用同类技术路线（即强化学习）的前沿云服务稳健性算法进行横向比较。这一缺失使得对本方法在相同技术范式下的创新性与性能优越性的评估不够充分，难以全面凸显其在解决“可靠性-成本”平衡问题上的独特价值。

为解决上述问题，并进一步夯实研究的严谨性与说服力，后续工作的重点将是：

系统性地检索并复现近年来发表于顶级学术会议的、基于强化学习的云服务容错与稳健性调度算法。计划将这些前沿算法作为新的对比基线，集成到现有的实验框架中，进行深入的横向性能比较。

六、参考文献

[1] Jamil, B., Ijaz, H., Shojafar, M., & Munir, K. (2023). IRATS: A DRL-based intelligent priority and deadline-aware online resource allocation and task scheduling algorithm in a vehicular fog network. Ad Hoc Networks, 141, 103090.

[2] M. Fan, X. Zhao, X. Zuo and L. Ye, "A Budget-Constrained Workflow Scheduling Approach With Priority Adjustment and Critical Task Optimizing in Clouds," in IEEE Transactions on Automation Science and Engineering, doi: 10.1109/TASE.2024.3456762.

[3] G. Fan, X. Chen, Z. Li, H. Yu and Y. Zhang, "An Energy-Efficient Dynamic Scheduling Method of Deadline-Constrained Workflows in a Cloud Environment," in IEEE Transactions on Network and Service Management, vol. 20, no. 3, pp. 3089-3103, Sept. 2023, doi: 10.1109/TNSM.2022.3228402.

[4] Mohammed Maray, Ehzaz Mustafa, Junaid Shuja, Muhammad Bilal, "Dependent task offloading with deadline-aware scheduling in mobile edge networks," in Internet of Things,Volume 23, 2023, doi: 100868/ISSN 2542-6605.

[5] L. Ye, Y. Xia, L. Yang and Y. Zhan, "Dynamic Scheduling Stochastic Multiworkflows With Deadline Constraints in Clouds," in IEEE Transactions on Automation Science and Engineering, vol. 20, no. 4, pp. 2594-2606, Oct. 2023, doi: 10.1109/TASE.2022.3204313.

[6] J. Jin, L. Hao, Y. Xu, J. Wu and Q. -S. Jia, "Joint Scheduling of Deferrable Demand and Storage With Random Supply and Processing Rate Limits," in IEEE Transactions on Automatic Control, vol. 66, no. 11, pp. 5506-5513, Nov. 2021, doi:

10.1109/TAC.2020.3046555.

[7] Sharif, Z., Jung, L.T., Ayaz, M., Yahya, M., & Pitafi, S. (2023). Priority-based task scheduling and resource allocation in edge computing for health monitoring system. J. King Saud Univ. Comput. Inf. Sci., 35, 544-559.

[8]J. X. Liao and X. W. Wu, "Resource Allocation and Task Scheduling Scheme in Priority-Based Hierarchical Edge Computing System," 2020 19th International Symposium on Distributed Computing and Applications for Business Engineering and Science (DCABES), Xuzhou, China, 2020, pp. 46-49, doi: 10.1109/DCABES50732.2020.00021.

[9] J. Liu et al., "Reliability-Enhanced Task Offloading in Mobile Edge Computing Environments," in IEEE Internet of Things Journal, vol. 9, no. 13, pp. 10382-10396, 1 July1, 2022, doi: 10.1109/JIOT.2021.3115807.

[10] M. Zhao, X. Zhang, Z. Meng and X. Hou, "Reliable DNN Partitioning for UAV Swarm," 2022 International Wireless Communications and Mobile Computing (IWCMC), Dubrovnik, Croatia, 2022, pp. 265-270, doi: 10.1109/IWCMC55113.2022.9824945.

[11] Wang, Z., Jin, L. & Yi, K. Intelligent hyperspectral target detection for reliable IoV applications. J Wireless Com Network 2022, 79 (2022). https://doi.org/10.1186/s13638-022-02161-z

[12] L. Chen, J. Qi, X. Su and R. Wang, "REMR: A Reliability Evaluation Method for Dynamic Edge Computing Network Under Time Constraint," in IEEE Internet of Things Journal, vol. 10, no. 5, pp. 4281-4291, 1 March1, 2023, doi: 10.1109/JIOT.2022.3216056.

[13] K. Peng, Y. Yang, S. Wang, P. Xiao and V. C. M. Leung, "Reliability-Aware

Proactive Offloading in Mobile Edge Computing Using Stackelberg Game Approach," in IEEE Internet of Things Journal, vol. 11, no. 9, pp. 16660-16671, 1 May1, 2024, doi: 10.1109/JIOT.2024.3354700.

[14] L. Dong, W. Wu, Q. Guo, M. N. Satpute, T. Znati and D. Z. Du, "Reliability-Aware Offloading and Allocation in Multilevel Edge Computing System," in IEEE Transactions on Reliability, vol. 70, no. 1, pp. 200-211, March 2021, doi: 10.1109/TR.2019.2909279.

[15] B. Ju, Y. Liu, L. Song, G. Gan, Z. Li and L. Jiang, "A High-Reliability Edge-Side Mobile Terminal Shared Computing Architecture Based on Task Triple-Stage Full-Cycle Monitoring," in IEEE Internet of Things Journal, vol. 10, no. 22, pp. 20149-20161, 15 Nov.15, 2023, doi: 10.1109/JIOT.2023.3283370.

[16] R. Huang, W. Wen, X. Chen, Z. Zhou, Q. Chen and C. Dong, "Joint Power Allocation and Task Replication for Reliability-Sensitive Services in NOMA-Enabled Vehicular Edge Computing," in IEEE Transactions on Vehicular Technology, vol. 73, no. 3, pp. 4178-4193, March 2024, doi: 10.1109/TVT.2023.3327073.

[17] Yan H, Zhu X, Chen H, et al. DEFT: Dynamic fault-tolerant elastic scheduling for tasks with uncertain runtime in cloud[J]. Information Sciences, 2019, 477: 30-46.

[18] M. Zhao, X. Zhang, Z. Meng and X. Hou, "Reliable DNN Partitioning for UAV Swarm," 2022 International Wireless Communications and Mobile Computing (IWCMC), Dubrovnik, Croatia, 2022, pp. 265-270, doi: 10.1109/IWCMC55113.2022.9824945.

[19] S. Bebortta, O. Pal, D. Senapati and S. S. Tripathy, "Cost-Aware Computation

Offloading for Managing Cloud-Bursts in IoT-Based Cloud-Fog Networks," 2024 1st International Conference on Cognitive, Green and Ubiquitous Computing (IC-CGU), Bhubaneswar, India, 2024, pp. 1-6, doi: 10.1109/IC-CGU58078.2024.10530817.

[20] K. J. Reddy, P. Udayaraju, V. P. R. Pamaiahgari and V. D. Kumar, "Implementing Resource-Aware Scheduling Algorithm for Improving Cost Optimization in Cloud Computing," 2025 4th International Conference on Sentiment Analysis and Deep Learning (ICSADL), Bhimdatta, Nepal, 2025, pp. 282-287, doi: 10.1109/ICSADL65848.2025.10933319.

[21] K. Lei, J. Wang, Y. Yu, W. Chen and L. Li, "Research on Heuristic Deployment and Scheduling Strategy of Cloud Resources Oriented to Cost Optimization," 2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT), Kharagpur, India, 2021, pp. 01-06, doi: 10.1109/ICCCNT51525.2021.9579835.

[22] Y. T. H. Hlaing and T. T. Yee, "Static Independent Task Scheduling on Virtualized Servers in Cloud Computing Environment," 2019 International Conference on Advanced Information Technologies (ICAIT), Yangon, Myanmar, 2019, pp. 55-59, doi: 10.1109/AITC.2019.8920865.

[23] S. Meng, L. Luo, X. Qiu and Y. Dai, "Service-Oriented Reliability Modeling and Autonomous Optimization of Reliability for Public Cloud Computing Systems," in IEEE Transactions on Reliability, vol. 71, no. 2, pp. 527-538, June 2022, doi: 10.1109/TR.2022.3154651.

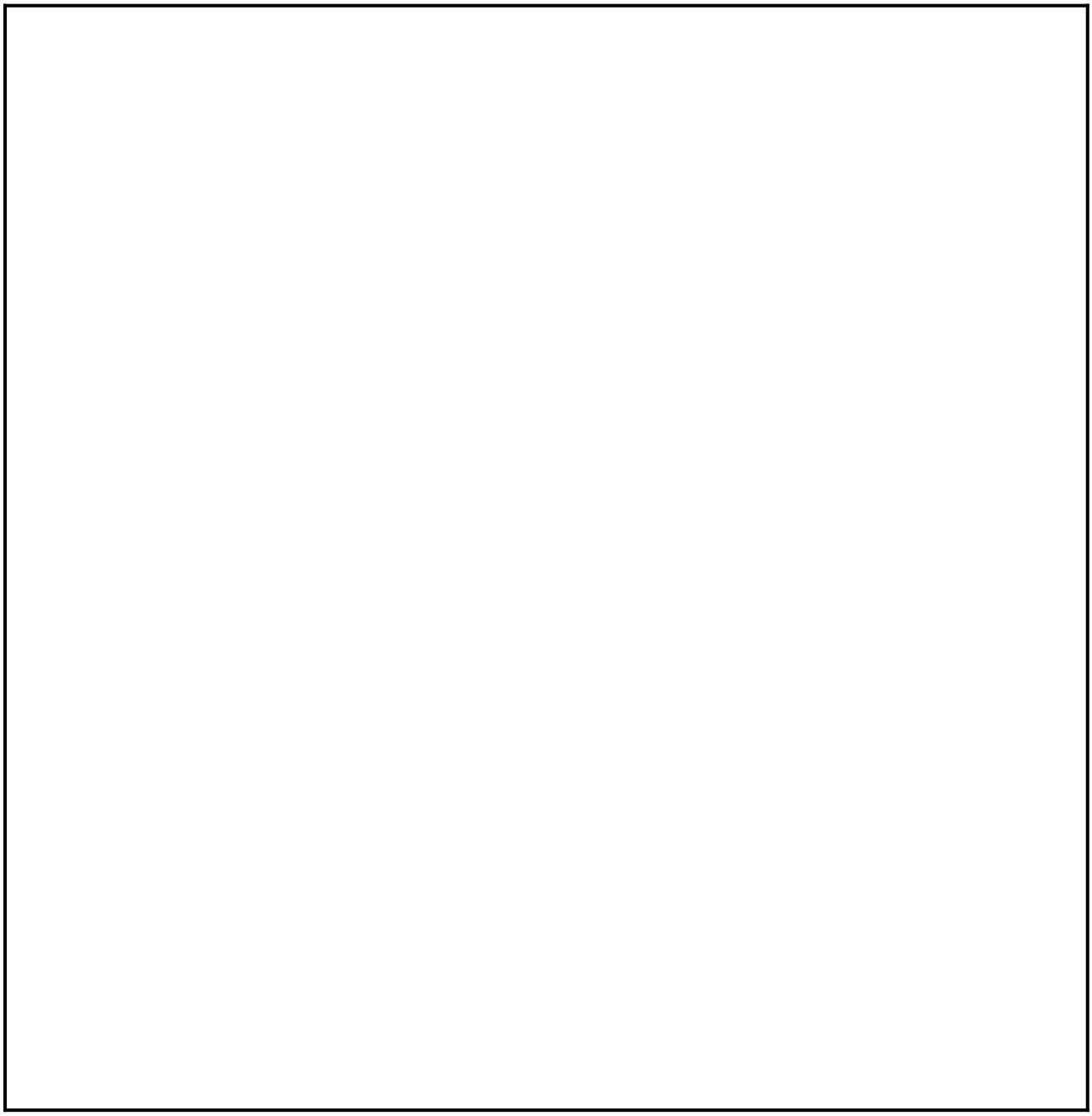
[24] X. Tang, Y. Liu, Z. Zeng and B. Veeravalli, "Service Cost Effective and Reliability

Aware Job Scheduling Algorithm on Cloud Computing Systems," in IEEE Transactions on Cloud Computing, vol. 11, no. 2, pp. 1461-1473, 1 April-June 2023, doi: 10.1109/TCC.2021.3137323.

[25] S. S. Gill and R. Buyya, "Failure Management for Reliable Cloud Computing: A Taxonomy, Model, and Future Directions," in Computing in Science & Engineering, vol. 22, no. 3, pp. 52-63, 1 May-June 2020, doi: 10.1109/MCSE.2018.2873866.

[26] Y. Alsenani, G. Crosby and T. Velasco, "SaRa: A Stochastic Model to Estimate Reliability of Edge Resources in Volunteer Cloud," 2018 IEEE International Conference on Edge Computing (EDGE), San Francisco, CA, USA, 2018, pp. 121-124, doi: 10.1109/EDGE.2018.00024.

[27] X. Duan and Z. Wang, "Uncertainty-Aware Scheduling of Hierarchical Realtime Workflows with Deadline Constraints in Cloud Environments," 2025 5th International Conference on Computer Science and Blockchain (CCSB), Shenzhen, China, 2025, pp. 209-212, doi: 10.1109/CCSB66722.2025.11154293.



导师评语
论文针对面向满意度和可靠性的云服务分级工作流在线调度方法的关键技术开展研究工作。论文研究内容充实，研究进度符合培养方案要求，后期研究计划安排合理，预计能够按时完成论文工作。 导师签字： 日期：　　年　　月　　日
考核结果
经学院组织评审，该生中期考核结果为： ☐通过　　☐未通过 学院负责人签字（盖章）：

日期：　　年　　月　　日